

**BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**



BÁO CÁO ĐỒ ÁN MÔN CƠ SỞ DỮ LIỆU PHÂN TÁN

**ĐỀ TÀI: MERKLE TREE LOG INTEGRITY: "IMMUTABLE
AUDIT TRAIL"**

Môn học: Cơ sở dữ liệu phân tán

Giảng viên hướng dẫn: Ths. Lê Hà Thanh

Thực hiện bởi sinh viên: Hồ Ngọc Hoàng Anh

Mã số sinh viên: N23DCCN071

TP.HCM, tháng /20...

Mục Lục

TÓM TẮT DỰ ÁN.....	4
CHƯƠNG I. GIỚI THIỆU.....	5
1.1. Lý do chọn đề tài.....	5
1.2. Mục tiêu của đề tài.....	5
1.3. Phạm vi và đối tượng nghiên cứu.....	5
1.4. Phương pháp thực hiện.....	5
1.5. Cấu trúc báo cáo.....	6
CHƯƠNG II. CƠ SỞ LÝ THUYẾT.....	7
2.1. Các khái niệm liên quan trong DDBMS (Özsu & Valduriez).....	7
2.2. Công nghệ, mô hình và thuật toán sử dụng.....	7
2.3. Các nghiên cứu hoặc hệ thống liên quan.....	7
CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	9
3.1. Khảo sát hệ thống và thu thập yêu cầu.....	9
3.2. Phân tích yêu cầu và kịch bản tấn công.....	9
3.3. Thiết kế tổng thể hệ thống (Kiến trúc & Luồng hoạt động).....	9
3.4. Thiết kế cơ sở dữ liệu SQLite.....	10
3.5. Thiết kế chức năng (Merkle Algorithm & Fault Tolerance).....	12
CHƯƠNG IV. XÂY DỰNG VÀ TRIỂN KHAI.....	13
4.1. Môi trường và công cụ phát triển.....	13
4.2. Mô tả các chức năng chính.....	13
4.3. Giao diện minh họa Dashboard.....	13
4.4. Cài đặt và cấu hình hệ thống.....	13
CHƯƠNG V. ĐÁNH GIÁ VÀ KẾT QUẢ.....	15

5.1. Kết quả đạt được (Forensics).....	15
5.2. Đánh giá ưu điểm (Benchmarks).....	15
5.3. Hạn chế của đề tài.....	15
CHƯƠNG VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	16
6.1. Kết luận.....	16
6.2. Hướng phát triển trong tương lai.....	16
TÀI LIỆU THAM KHẢO.....	17
PHỤ LỤC.....	18
A. Hướng dẫn chạy thử nghiệm nhanh qua CLI:.....	18
1. Dọn dẹp và khởi tạo dữ liệu sạch ban đầu:.....	18
2. Thực hiện tấn công giả lập:.....	18
3. Chạy công cụ kiểm toán và pháp y số:.....	18
B. Mã nguồn cốt lõi dựng cây Merkle (`core/merkle.py`):.....	18

TÓM TẮT DỰ ÁN

Đề án tập trung giải quyết bài toán bảo vệ tính toàn vẹn của nhật ký giao dịch ngân hàng phân tán chống lại mối đe dọa tấn công nội bộ (Insider Attack). Hệ thống sử dụng cấu trúc dữ liệu cây Merkle (Merkle Tree) để tạo ra dấu vân tay mật mã (Root Hash) cho từng khối giao dịch và đăng ký lên Bên thứ ba tin cậy (TTP). Khi xảy ra can thiệp vật lý vào CSDL thô tại chi nhánh (Site B), hệ thống sẽ đối chiếu Root Hash cục bộ với TTP để phát hiện tức thời, đồng thời chạy giải thuật so khớp Leaf Hash với bản sao sạch (Site A) nhằm chỉ điểm và điều tra pháp lý số (Forensics) chính xác bản ghi bị sửa đổi, xóa hoặc chen trái phép. Hệ thống được triển khai dưới dạng 4 node microservices REST API giao tiếp qua HTTP và tích hợp giao diện Web Dashboard quản trị sinh động.

CHƯƠNG I. GIỚI THIỆU

1.1. Lý do chọn đề tài

Trong kỷ nguyên số hóa ngân hàng và hệ quản trị cơ sở dữ liệu phân tán, nhật ký giao dịch (Audit Trail/Transaction Log) là dữ liệu nhạy cảm hàng đầu, đóng vai trò chứng cứ pháp lý và đối chiếu tài chính. Tuy nhiên, các cơ chế tường lửa, API Gateway hay phân quyền ứng dụng chỉ ngăn chặn được các hacker bên ngoài, hoàn toàn bất lực trước **Tấn công nội bộ (Insider Attacks)**.

Một Quản trị viên cơ sở dữ liệu (Rogue DBA) có quyền root tại chi nhánh ngân hàng (Site B) hoàn toàn có thể mở tệp CSDL vật lý dưới đĩa cứng để chạy lệnh SQL sửa đổi số tiền, xóa giao dịch hoặc chèn giao dịch không nhằm rút ruột tài sản ngân hàng. Để phát hiện và chỉ điểm được hành vi phá hoại này mà không tốn nhiều băng thông truyền file qua mạng, giải pháp ứng dụng Cây mật mã Merkle kết hợp Bên thứ ba tin cậy (TTP) được lựa chọn nghiên cứu và triển khai trong đồ án này.

1.2. Mục tiêu của đề tài

- Hiện thực hóa mô hình bảo vệ tính toàn vẹn nhật ký giao dịch phân tán cho Đề tài 105.
- Lập trình thuật toán xây dựng cây Merkle, tính toán mã băm gốc (Root Hash) và sinh bằng chứng kiểm toán (Merkle Proof) thủ công không qua thư viện ngoài.
- Thiết kế hệ thống phân tán gồm 4 node độc lập chạy trên localhost để kiểm thử cơ chế nhân bản đồng bộ (ROWA) và khả năng chịu lỗi sập mạng.
- Phát triển công cụ tự động phát hiện, định vị lỗi pháp y (Forensics) và trực quan hóa toàn bộ quá trình qua Web Dashboard.

1.3. Phạm vi và đối tượng nghiên cứu

- **Đối tượng nghiên cứu:** Cấu trúc dữ liệu Merkle Tree, hàm băm SHA-256, hệ quản trị cơ sở dữ liệu phân tán, các giao thức nhân bản dữ liệu (ROWA) và kiểm soát toàn vẹn dữ liệu.
- **Phạm vi nghiên cứu:** Giả lập nhật ký giao dịch ngân hàng ngân quỹ của 2 chi nhánh độc lập dưới dạng SQLite vật lý trên Windows, giao tiếp qua giao thức mạng HTTP/REST API.

1.4. Phương pháp thực hiện

- **Nghiên cứu lý thuyết:** Đọc và áp dụng lý thuyết từ giáo trình **Principles of Distributed Database Systems** (Özsu & Valduriez) về điều khiển dữ liệu phân tán, nhân bản dữ liệu và tính nhất quán.
- **Lập trình thực nghiệm:** Sử dụng ngôn ngữ Python xây dựng các Flask microservices; sử dụng SQLite3 cho CSDL cục bộ; thiết kế giao diện Single Page ứng dụng HTML/CSS/JS thuần vẽ cây nhị phân bằng Canvas HTML5.

- **Kiểm thử đo đạc:** Chạy các kịch bản tấn công (Update, Delete, Insert) và đo đạc các chỉ số thời gian dựng cây, chi phí lưu trữ metadata.

1.5. Cấu trúc báo cáo

Báo cáo đồ án môn học gồm 6 chương:

- **Chương I. Giới thiệu:** Trình bày lý do chọn đề tài, mục tiêu, phạm vi và phương pháp thực hiện.
- **Chương II. Cơ sở lý thuyết:** Trình bày các kiến thức cốt lõi về DDBMS theo sách giáo khoa và cấu trúc Merkle Tree.
- **Chương III. Phân tích và Thiết kế hệ thống:** Khảo sát yêu cầu, thiết kế kiến trúc, lược đồ CSDL và giải thuật kiểm toán, chịu lỗi.
- **Chương IV. Xây dựng và Triển khai:** Chi tiết về môi trường, các chức năng chính, giao diện và hướng dẫn cài đặt chạy thử.
- **Chương V. Đánh giá và Kết quả:** Phân tích thực nghiệm kiểm toán, các chỉ số benchmark hiệu năng và hạn chế hệ thống.
- **Chương VI. Kết luận và Hướng phát triển:** Tổng kết đồ án và đề xuất hướng mở rộng.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT

2.1. Các khái niệm liên quan trong DDBMS (Özsu & Valduriez)

Dựa trên lý thuyết Hệ CSDL phân tán của Özsu & Valduriez, đồ án áp dụng các nguyên lý sau:

- **Distributed DBMS:** Hệ thống quản lý các cơ sở dữ liệu có quan hệ logic được phân tán trên mạng. Dự án phân rã dữ liệu giao dịch trên các Site vật lý riêng biệt ('site_a.db' và 'site_b.db').
- **Data Replication (Nhân bản dữ liệu):** Nhằm tăng tính sẵn sàng và chịu lỗi. Đồ án áp dụng chiến lược nhân bản đồng bộ **Eager Replication** kết hợp giao thức **ROWA (Read-One/Write-All)**. Giao dịch mới bắt buộc phải ghi thành công đồng thời lên tất cả các site bản sao thì mới được phản hồi thành công.
- **Mutual Consistency & Transactional Consistency (Tính nhất quán):** Hệ thống yêu cầu trạng thái dữ liệu trên các bản sao phải đồng nhất tuyệt đối (Strong Consistency) và tuân thủ thuộc tính 1-Copy Serializability (1SR). Khi DBA can thiệp thô sửa dữ liệu ở một site, tính nhất quán bị phá vỡ, kích hoạt cơ chế kiểm toán phát hiện.
- **Transaction Log / Write-Ahead Log (WAL):** Là tập tin ghi lại lịch sử thay đổi để đảm bảo tính bền vững (Durability) của ACID. Cây Merkle được xây dựng dựa trên cấu trúc nhật ký giao dịch nối tiếp (Append-only) này.
- **Semantic Integrity Control (Kiểm soát toàn vẹn ngữ nghĩa):** Đồ án hiện thực hóa cơ chế phát hiện vi phạm ràng buộc toàn vẹn động (Semantic Control) thông qua hệ thống kiểm toán độc lập sau khi dữ liệu đã được ghi xuống ổ đĩa (Auditing/Detection).

2.2. Công nghệ, mô hình và thuật toán sử dụng

- **Hàm băm Cryptographic Hash Function (SHA-256):** Áp dụng tính chất một chiều (one-way) và chống đụng độ (collision resistance) để tạo vân tay số cố định 256-bit đại diện cho giao dịch.
- **Cấu trúc dữ liệu Merkle Tree:** Cây nhị phân băm mật mã. Các node lá chứa hash của giao dịch thô. Node cha được tính toán bằng cách kết hợp mã băm của hai con trực tiếp:

$$ParentHash = SHA256(LeftHash + RightHash)$$

Root Hash ở đỉnh cây đại diện cho toàn bộ các giao dịch nằm trong khối. Thay đổi bất kỳ 1 byte dữ liệu ở lá sẽ dẫn đến sự thay đổi lan truyền (Avalanche effect) làm thay đổi hoàn toàn Root Hash.

- **Trusted Third Party (TTP):** Bên thứ ba trung lập lưu trữ Root Hash để đối chứng tính toàn vẹn dữ liệu của các site chi nhánh, đóng vai trò như một chứng từ số bất biến mà chi nhánh không thể chối bỏ hay làm giả.

2.3. Các nghiên cứu hoặc hệ thống liên quan

Trong thực tế, mô hình cây Merkle kết hợp TTP được ứng dụng rộng rãi trong các hệ thống đòi hỏi tính minh bạch cao như:

- **Blockchain:** Cấu trúc Merkle Tree nằm ở phần Header của mỗi block trong mạng Bitcoin, Ethereum để xác thực giao dịch nhanh mà không cần tải toàn bộ blockchain.
- **Certificate Transparency:** Hệ thống lưu trữ nhật ký chứng chỉ SSL/TLS công khai để phát hiện các chứng chỉ giả mạo hoặc cấp phát sai quy trình.
- **Git Version Control:** Quản lý lịch sử commit và tệp tin của các lập trình viên bằng cây Hash (Merkle DAG).

CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Khảo sát hệ thống và thu thập yêu cầu

Tại các ngân hàng thương mại, các chi nhánh địa phương (Site B) lưu trữ dữ liệu cục bộ để đảm bảo tốc độ phản hồi nhanh cho khách hàng. Tuy nhiên, do tập tin CSDL được lưu tại ổ cứng chi nhánh, quản trị viên database (DBA) có quyền truy cập root hệ điều hành hoàn toàn có thể lách qua phần mềm ứng dụng để chỉnh sửa số tiền trực tiếp trong đĩa. Hệ thống yêu cầu:

1. Phát hiện tức thời nếu có bất kỳ can thiệp thô nào vào dữ liệu giao dịch chi nhánh.
2. Tiết kiệm tối đa băng thông, không được truyền tải tập tin database hàng triệu bản ghi qua mạng để đối chứng.
3. Chỉ điểm chính xác mã giao dịch bị can thiệp và chỉ ra trường thông tin bị thay đổi.
4. Đảm bảo bảo mật và quyền riêng tư: Bên thứ ba tin cậy (TTP) kiểm toán không được phép đọc dữ liệu thô (tài khoản, số tiền) của khách hàng.

3.2. Phân tích yêu cầu và kịch bản tấn công

- **Phát hiện can thiệp (Detection):** So khớp Root Hash tính toán từ Site B với Root Hash gốc đăng ký tại TTP.

- **Định vị lỗi (Forensics & Localization):** So khớp Leaf Hashes giữa Site A (Replica sạch đối chứng) và Site B (Replica bị can thiệp) để tìm ra giao dịch lỗi.

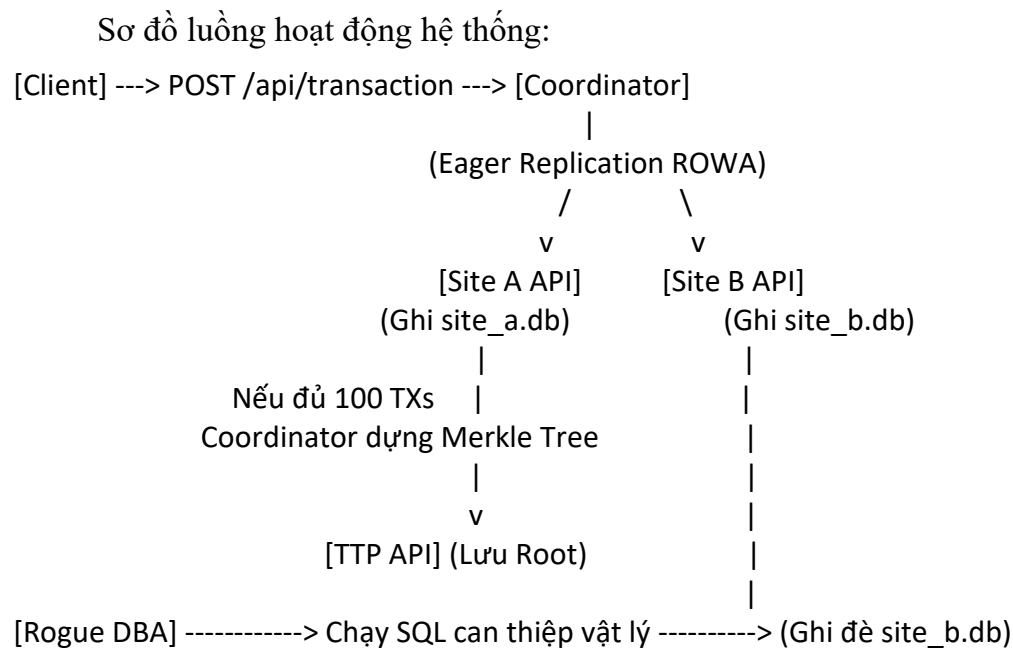
- **Kịch bản tấn công:** Giả lập 3 kiểu tấn công nội bộ thô tại Site B:

1. ***Update Attack:*** Sửa đổi trường `Amount` của một giao dịch cụ thể từ `\$100.00` thành `\$1000.00`.
2. ***Delete Attack:*** Xóa bỏ một giao dịch ra khỏi CSDL nhằm chối bỏ trách nhiệm.
3. ***Insert Attack:*** Chèn một giao dịch không (giao dịch ma) để rút tiền mặt.

3.3. Thiết kế tổng thể hệ thống (Kiến trúc & Luồng hoạt động)

Kiến trúc hệ thống bao gồm 4 Node Flask microservices phân tán chạy độc lập trên localhost:

1. **Coordinator (Port 5000):** Cổng giao tiếp trung tâm. Tiếp nhận giao dịch, nhân bản đồng bộ (ROWA), tự động đóng block (Block Size = 100) để dựng cây băm và đăng ký Root Hash lên TTP. Cung cấp Dashboard quản lý.
2. **Site A (Port 5001):** Lưu trữ CSDL bản sao sạch làm mốc đối chứng kiểm toán.
3. **Site B (Port 5002):** Lưu trữ CSDL chi nhánh chi tiết, mục tiêu giả lập tấn công.
4. **TTP (Port 5003):** Bên thứ ba trung lập lưu trữ Root Hash bất biến cấp block để kiểm toán.



3.4. Thiết kế cơ sở dữ liệu SQLite

Hệ thống sử dụng các tệp tin SQLite vật lý độc lập mô phỏng các site lưu trữ phân tán thực tế:

•Bảng `Banking\_Transactions` (Lưu tại Site A và Site B):

```

CREATE TABLE Banking_Transactions (

  TransactionID TEXT PRIMARY KEY,

  From_Account TEXT NOT NULL,

  To_Account TEXT NOT NULL,

  Amount REAL NOT NULL,

  Timestamp TEXT NOT NULL,

  BlockID INTEGER NOT NULL

);
  
```

•Bảng `Block\_Hashes` (Lưu tại node TTP):

```

CREATE TABLE Block_Hashes (

  BlockID INTEGER PRIMARY KEY,

  StartTxID TEXT NOT NULL,
  
```

EndTxID TEXT NOT NULL,

RootHash TEXT NOT NULL,

Timestamp TEXT NOT NULL

);

3.5. Thiết kế chức năng (Merkle Algorithm & Fault Tolerance)

A. Thuật toán Cây Merkle (Merkle Engine)

- **Chuẩn hóa lá (Leaf Serialization):** Mỗi giao dịch được chuẩn hóa thành chuỗi định dạng nghiêm ngặt để đảm bảo mã băm không bị lệch pha khi chuyển đổi môi trường:

$S(tx) = \text{TransactionID} \mid \text{From_Account} \mid \text{To_Account} \mid \text{Amount (2 chữ số thập phân)} \mid \text{Timestamp}$

$$\text{LeafHash} = \text{SHA256}(S(tx))$$

- **Xử lý nút lá lẻ:** Nếu tổng số lượng nút ở tầng hiện tại là số lẻ, thuật toán sao chép (duplicate) nút cuối cùng để thực hiện ghép cặp cân bằng.
- **Tính nút cha:** Ghép cặp hai mã băm con rồi băm kết hợp.

B. Quy trình Pháp y số (Forensic Process)

Nếu phát hiện lệch Root Hash, Detector sẽ gọi API để tải danh sách mã băm lá (Leaf Hashes) của Block đó từ Site A và Site B rồi tiến hành so khớp:

- **MODIFIED:** Cùng 'TransactionID' nhưng giá trị hash khác nhau. Đối chiếu dữ liệu chi tiết giữa Site A và Site B để chỉ ra trường thông tin bị sửa đổi (ví dụ: trường 'Amount' bị sửa đổi).
- **DELETED:** 'TransactionID' có trong danh sách băm của Site A nhưng không tồn tại ở Site B.
- **INJECTED:** 'TransactionID' có trong danh sách băm của Site B nhưng không tồn tại ở Site A.

C. Thiết kế Chịu lỗi phân tán (Fault Tolerance)

- **Kịch bản sập Site B (hoặc Site A):** Khi một site bị sập, Coordinator nhận lỗi kết nối mạng (ConnectionError). Theo giao thức ROWA (Write-All), Coordinator lập tức hủy giao dịch ghi mới (Rollback) và trả về mã lỗi HTTP 500 cho Client nhằm bảo vệ tính nhất quán dữ liệu tuyệt đối giữa các chi nhánh (Consistency over Availability). Luồng Đọc vẫn hoạt động bình thường bằng cách chuyển hướng đọc sang site còn lại (Site A).
- **Kịch bản sập TTP:** Các giao dịch lẻ vẫn ghi bình thường. Khi đến giao dịch thứ 100 cần đóng block, Coordinator không thể gửi Root Hash lên TTP do TTP sập. Giao dịch đóng block này sẽ bị Coordinator từ chối và báo lỗi 500 để đảm bảo tính an toàn hệ thống. Quá trình kiểm toán (Audit) cũng bị chặn hoàn toàn vì thiếu nguồn đối chứng.

CHƯƠNG IV. XÂY DỰNG VÀ TRIỂN KHAI

4.1. Môi trường và công cụ phát triển

- **Hệ điều hành:** Windows 10/11.
- **Ngôn ngữ phát triển:** Python 3.10.
- **Công cụ CSDL:** SQLite3.
- **Môi trường ảo:** Python virtualenv (`.venv`).
- **Các thư viện chính:** Flask, Requests, Chart.js, Vanilla JS Canvas.

4.2. Mô tả các chức năng chính

- **`generator.py` (Seeding):** Dọn dẹp dữ liệu cũ, sinh 500 giao dịch mẫu sạch chia đều làm 5 block (mỗi block 100 giao dịch) và đăng ký Root Hash tương ứng lên TTP.
- **`attack.py` (Simulate Attack):** Giả lập Rogue DBA sửa tiền, xóa hoặc chèn giao dịch trực tiếp bằng SQL thô trong database vật lý `site\_b.db`.
- **`detector.py` (Audit CLI):** Chạy quy trình kiểm toán từ Terminal bằng cách so khớp Root Hash với TTP và chạy thuật toán so khớp Leaf Hash đối chiếu Site A để chỉ điểm sai khác.
- **Web Dashboard (Coordinator Node):** Cung cấp giao diện đồ họa trực quan hóa danh sách giao dịch, cây băm Merkle tương tác trực tiếp bằng HTML5 Canvas và Tab đo đặc hiệu năng.

4.3. Giao diện minh họa Dashboard

Giao diện Web Dashboard thiết kế theo ngôn ngữ hiện đại Glassmorphism:

- **Trang chủ:** Hiện thị danh sách giao dịch phân mảnh của Site B, bảng thêm giao dịch mới và đèn trạng thái kết nối 4 Node mạng (Coordinator, Site A, Site B, TTP).
- **Cây Merkle trực quan (Canvas):** Click vào từng Block để vẽ cây nhị phân Merkle. Khi chạy Detector, nếu phát hiện can thiệp, Block bị tampered sẽ chuyển sang **màu đỏ**, toàn bộ đường dẫn liên kết từ lá bị hack lên đỉnh Root Hash cũng được vẽ bằng **màu đỏ** để thấy cô dễ dàng quan sát vết lan truyền băm. Bảng báo lỗi chi tiết sẽ chỉ điểm cụ thể TransactionID bị can thiệp.
- **Tab Đo đặc hiệu năng:** Biểu đồ Chart.js trực quan hóa thời gian dựng cây Merkle (Build Time) và chi phí lưu trữ phát sinh (Storage Overhead).

4.4. Cài đặt và cấu hình hệ thống

Các bước thiết lập và khởi chạy trên hệ điều hành Windows:

1. Bước 1: Khởi tạo và kích hoạt môi trường ảo:

```
python -m venv .venv
```

```
.\.venv\bin\Activate.ps1
```

2. Bước 2: Cài đặt các thư viện phụ thuộc:

```
pip install -r requirements.txt
```

3. Bước 3: Khởi chạy toàn bộ hệ thống (4 Nodes):

```
python run_all.py
```

(Tiến trình sẽ lắng nghe các cổng: 5000 - Coordinator, 5001 - Site A, 5002 - Site B, 5003 - TTP).

4. **Bước 4: Truy cập Dashboard:** Mở trình duyệt Web tại địa chỉ `http://127.0.0.1:5000`.

CHƯƠNG V. ĐÁNH GIÁ VÀ KẾT QUẢ

5.1. Kết quả đạt được (Forensics)

Hệ thống thực hiện kiểm toán và phát hiện chính xác 100% các cuộc tấn công giả lập:

- **UPDATE:** Giả lập sửa số tiền giao dịch 'TX-100150' ở Block 2 tại Site B từ \$100.00 thành \$1000.00. Detector phát hiện Block 2 bị can thiệp, chỉ điểm đúng ID 'TX-100150', trường bị sửa là 'Amount', giá trị sạch tại Site A là '100.00', giá trị giả mạo tại Site B là '1000.00'.
- **DELETE:** Giả lập xóa giao dịch 'TX-100120' ở Block 2 tại Site B. Detector báo lỗi cấu trúc cây ('row_count_mismatch') và báo cáo giao dịch 'TX-100120' bị xóa mất tích.
- **INSERT:** Giả lập chèn giao dịch ma 'TX-FAKE' vào Block 3 tại Site B. Detector lập tức báo cáo phát hiện giao dịch lạ chèn không là 'TX-FAKE' ở Block 3.

5.2. Đánh giá ưu điểm (Benchmarks)

Dữ liệu đo đạc thực nghiệm từ script 'scripts/benchmark.py' chứng minh tính khả thi vượt trội:

- **Thời gian dựng cây (Build Time):** Cực kỳ nhanh. Chỉ mất **0.27 ms** để dựng cây Merkle cho 100 giao dịch; mất **6.08 ms** cho block 2000 giao dịch. Tốc độ dựng cây tăng trưởng tuyến tính $O(N)$.
- **Thời gian xác thực (Verify Time):** Nhờ cấu trúc cây nhị phân băm, thời gian xác thực chỉ tốn khoảng **0.01 ms** (độ phức tạp $O(\log N)$).
- **Chi phí lưu trữ siêu dữ liệu (Storage Overhead):** Vô cùng tiết kiệm. Với block 100 giao dịch (dữ liệu thô ~7 KB), TTP chỉ cần lưu trữ **64 bytes** Root Hash (overhead ~1%). Kích thước block càng lớn, tỷ lệ overhead càng nhỏ tiệm cận về mức 0.05%, giúp tiết kiệm tối đa tài nguyên đĩa cứng.
- **Tiết kiệm băng thông mạng:** Việc kiểm toán định kỳ chỉ cần truyền chuỗi băm 64 ký tự (Root Hash) qua mạng thay vì truyền tải toàn bộ file database gốc, tránh tắc nghẽn băng thông.

5.3. Hạn chế của đề tài

- **Single Point of Failure (SPOF):** Node Coordinator hiện tại là điểm lỗi duy nhất của luồng ghi dữ liệu và dựng cây băm. Node TTP cũng là SPOF của luồng kiểm toán.
- **Bảo mật bản sao sạch:** Cơ chế định vị giao dịch lỗi dựa trên giả định CSDL Site A là hoàn toàn sạch và tin cậy. Nếu kẻ tấn công chiếm quyền quản trị root và sửa đổi đồng bộ cả Site A, Site B lẫn TTP, hệ thống kiểm toán sẽ bị vô hiệu hóa.

CHƯƠNG VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đồ án đã hiện thực hóa thành công giải pháp bảo vệ tính toàn vẹn nhật ký giao dịch ngân hàng phân tán bằng cây Merkle theo đúng yêu cầu của Đề tài 105. Hệ thống kết hợp nhuần nhuyễn giữa lý thuyết cơ sở dữ liệu phân tán (Eager replication, ROWA, CAP Theorem) và các cấu trúc mật mã an toàn. Kết quả thực nghiệm cho thấy giải pháp đạt hiệu năng cao, thời gian kiểm tra cực nhanh, chi phí lưu trữ thấp và bảo vệ hiệu quả quyền riêng tư của thông tin khách hàng tại bên thứ ba TTP.

6.2. Hướng phát triển trong tương lai

- **Tích hợp thuật toán đồng thuận phân tán:** Phát triển cụm Coordinator chạy giao thức đồng thuận **Raft** hoặc **PBFT** để tự động bầu chọn node điều phối mới khi xảy ra sự cố, loại bỏ hoàn toàn lỗi SPOF.
- **Ứng dụng Blockchain:** Lưu trữ Root Hash trên một sổ cái Blockchain phi tập trung (ví dụ: Hyperledger Fabric) để đảm bảo Root Hash của khối không thể bị sửa đổi ngay cả khi kẻ tấn công chiếm quyền máy chủ TTP.
- **Chữ ký số (Digital Signatures):** Yêu cầu mỗi chi nhánh ký số lên giao dịch trước khi thực hiện replicate để chống chối bỏ ở mức độ cao hơn.

TÀI LIỆU THAM KHẢO

- [1] M. Tamer Özsu & Patrick Valduriez, *Principles of Distributed Database Systems*, 4th Edition, Springer, 2020.
- [2] Slide bài giảng môn học Cơ sở dữ liệu phân tán, Học viện Công nghệ Bưu chính Viễn thông.
- [3] Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", 2008.

PHỤ LỤC

A. Hướng dẫn chạy thử nghiệm nhanh qua CLI:

1. Dọn dẹp và khởi tạo dữ liệu sạch ban đầu:

```
python scripts/generator.py
```

2. Thực hiện tấn công giả lập:

```
python scripts/attack.py --type update --transaction-id TX-100150 --amount 1000
```

3. Chạy công cụ kiểm toán và pháp y số:

```
python scripts/detector.py
```

B. Mã nguồn cốt lõi dựng cây Merkle (`core/merkle.py`):

```
import hashlib

from typing import List, Dict, Any

class MerkleNode:

    def __init__(self, hash_val: str, left=None, right=None, tx_id: str = None):

        self.hash = hash_val

        self.left = left

        self.right = right

        self.tx_id = tx_id

class MerkleTree:

    def __init__(self, transactions: List[Dict[str, Any]]):

        self.leaves = [self._create_leaf(tx) for tx in transactions]

        self.root = self._build_tree(self.leaves)

    def _create_leaf(self, tx: Dict[str, Any]) -> MerkleNode:
```

Chuẩn hóa dữ liệu giao dịch trước khi băm

```
tx_str = f'{tx["TransactionID"]}||{tx["From_Account"]}||{tx["To_Account"]}||{tx["Amount"]:.2f}||{tx["Timestamp"]}'
```

```
hash_val = hashlib.sha256(tx_str.encode('utf-8')).hexdigest()
```

```
return MerkleNode(hash_val, tx_id=tx["TransactionID"])
```

```
def _build_tree(self, nodes: List[MerkleNode]) -> MerkleNode:
```

```
    if not nodes:
```

```
        return MerkleNode(hashlib.sha256(b'').hexdigest())
```

```
    current_level = nodes
```

```
    while len(current_level) > 1:
```

```
        next_level = []
```

```
        for i in range(0, len(current_level), 2):
```

```
            left = current_level[i]
```

```
            if i + 1 < len(current_level):
```

```
                right = current_level[i+1]
```

```
            else:
```

```
                right = left # Sao chép nút nếu số lượng nút lẻ
```

```
            combined_hash = hashlib.sha256((left.hash + right.hash).encode('utf-8')).hexdigest()
```

```
            parent = MerkleNode(combined_hash, left, right)
```

```
            next_level.append(parent)
```

```
        current_level = next_level
```

```
    return current_level[0]
```

```
def get_root_hash(self) -> str:
```

```
    return self.root.hash
```